

INFORME DE VALIDACIÓN DE IMPLEMENTACIÓN

Componente	Clúster Kafka — Backbone de eventos (KRaft, sin Zookeeper)
Clúster / Entorno	Kafka Principal (DC Principal)
Nodos validados	pbigd-kaf01 · pbigd-kaf02 · pbigd-kaf03 (rol: broker KRaft, mismo rol en los 3)
Versión del software	Apache Kafka 4.0.1 (modo KRaft, sin Zookeeper)
SO / Runtime	Rocky Linux 10.2 (Red Quartz) · Podman 5.8.2 · systemd · OpenJDK 21.0.8 (funcional, no homologado)
Fecha de validación	09 de julio de 2026
Resultado general	✗ NO APTO / RECHAZADO

1. RESUMEN EJECUTIVO

Se realizó la validación de la implementación del Clúster Kafka Principal del proyecto Big Data de la Cooperativa Jardín Azuayo (proyecto Bluepoint). La validación se ejecutó el 09 de julio de 2026 entre las 16:07 y las 16:08 en los 3 nodos del clúster (pbigd-kaf01, pbigd-kaf02, pbigd-kaf03), todos con el mismo rol de broker KRaft, mediante el script `validate_kafka_principal.sh`, cubriendo 8 módulos de verificación con 40 puntos de control por nodo (120 en total).

La infraestructura base (sistema operativo, Podman, persistencia en disco, red y observabilidad) está correctamente desplegada y es homogénea en los 3 nodos. Sin embargo, el clúster **no opera como un clúster KRaft de 3 nodos**: cada broker responde de forma individual, pero solo se reconoce a sí mismo dentro del quórum de metadata, y la creación de un topic con factor de replicación 3 falla porque nunca hay más de 1 broker registrado. La causa raíz, confirmada con evidencia (ver §3 y §4.4), es que los 3 nodos se formatearon con un `cluster.id` distinto cada uno, por lo que operan como 3 mini-clústeres aislados de 1 nodo en lugar de un único clúster de 3. Esto impide la función core validada (alta disponibilidad con RF=3) y bloquea el paso a producción hasta remediarse.

102	9	9
PASS	WARN	FAIL

2. MATRIZ DE ESTADO POR NODO

Los 3 nodos cumplen el mismo rol (broker KRaft) y ejecutan exactamente el mismo módulo de checks. Los resultados fueron **idénticos en los 3 nodos** en todas las verificaciones — no se observó divergencia entre ellos. Las verificaciones de conectividad entre pares (ping / TCP 9092 / TCP 9093 hacia los otros 2 nodos, sección 4.3 del script) se muestran consolidadas en una sola fila por tipo de chequeo, dado que las 6 combinaciones nodo-vecino evaluadas por sección resultaron uniformemente PASS.

Verificación	pbigd-kaf01	pbigd-kaf02	pbigd-kaf03
SO: Rocky Linux 10.2 (Red Quartz)	PASS	PASS	PASS
Kernel 6.12.0-124.x.el10_1.x86_64	PASS	PASS	PASS
Podman ≥5.x (5.8.2 instalado)	PASS	PASS	PASS
systemd: running	PASS	PASS	PASS
NTP sincronizado (crítico para consenso KRaft)	PASS	PASS	PASS
Naming convention hostname (pbigd-kaf<NN>)	PASS	PASS	PASS
/data/kafka existe (owner/perms/uso disco)	PASS	PASS	PASS
/data/kafka filesystem xfs (no overlay)	PASS	PASS	PASS
/var/log/kafka existe	PASS	PASS	PASS
Partición /data independiente montada	PASS	PASS	PASS
KAFKA_HOME (/opt/kafka) dentro del contenedor	PASS	PASS	PASS
Unidad systemd con nombre 'kafka'	WARN	WARN	WARN
Contenedor Podman Kafka en estado Up (kafka-broker-0N)	PASS	PASS	PASS
Naming convention del contenedor (kafka-broker-N)	PASS	PASS	PASS
Política de reinicio del contenedor (recomendado: always)	WARN	WARN	WARN
Puerto 9092 (broker/client) escuchando localmente	PASS	PASS	PASS
Puerto 9093 (controller KRaft) escuchando localmente	PASS	PASS	PASS
Ping a los otros 2 nodos del clúster	PASS	PASS	PASS
TCP 9092 a los otros 2 nodos del clúster	PASS	PASS	PASS
TCP 9093 (controller) a los otros 2 nodos del clúster	PASS	PASS	PASS
Sin puerto 2181 (Zookeeper) activo — consistente con KRaft	PASS	PASS	PASS
Sin proceso Zookeeper dentro del contenedor	PASS	PASS	PASS
Quórum KRaft respondiendo (kafka-metadata-quorum.sh)	PASS	PASS	PASS

Brokers registrados ≥3 en el clúster	FAIL	FAIL	FAIL
Ciclo de topic de prueba: crear → producir → consumir → borrar	FAIL	FAIL	FAIL
Particiones sub-replicadas	PASS	PASS	PASS
Quórum de brokers 3/3 requerido en Principal	FAIL	FAIL	FAIL
TCP broker port (9092) en cada nodo del clúster	PASS	PASS	PASS
Nodos del clúster Principal 3/3 — completo (capa TCP)	PASS	PASS	PASS
Logs presentes en /var/log/kafka	PASS	PASS	PASS
Sin errores críticos en últimos 100 logs del contenedor	PASS	PASS	PASS
Grafana Alloy activo	PASS	PASS	PASS
node-exporter puerto 9100	WARN	WARN	WARN
Versión Kafka: 4.0.1 (esperada: 4.0.1)	PASS	PASS	PASS
JDK dentro del contenedor compatible con Kafka 4.x	PASS	PASS	PASS

Total por nodo (según resumen del propio script): 40 checks / 34 PASS / 3 FAIL / 3 WARN — idéntico en los 3 nodos. Filas consolidadas: conectividad entre pares (6 combinaciones nodo–vecino en sección 4.3, 3 combinaciones en sección 6.3), todas PASS.

3. HALLAZGOS Y ACCIONES CORRECTIVAS

Tipo	Hallazgo	Detalle técnico	Acción requerida
FAIL	Brokers activos insuficientes (1 de 3 esperados)	<code>kafka-broker-api-versions.sh</code> reporta 1 solo broker respondiendo a API versions, en los 3 nodos evaluados por igual. Es el síntoma central: los 3 nodos no se ven entre sí como parte del mismo clúster.	OBLIGATORIO — ver acción obligatoria #1 en §5 (reformatear KRaft storage con <code>cluster.id</code> compartido).
FAIL	Creación de topic de prueba falla (RF=3 no alcanzable)	En los 3 nodos: <code>InvalidReplicationFactorException: Unable to replicate the partition 3 time(s): The target replication factor of 3 cannot be reached because only 1 broker(s) are registered.</code> Es consecuencia directa del hallazgo anterior — Kafka rechaza correctamente el topic porque no hay 3 brokers físicos donde ubicar las réplicas.	OBLIGATORIO — se resuelve con la misma acción obligatoria #1 en §5; no requiere remediación independiente.
FAIL	Quórum de brokers 1/3 —	<code>kafka-metadata-quorum.sh describe --status</code> confirma 1/3 en los 3 nodos. El	OBLIGATORIO — mismo origen y misma acción

	clúster degradado	campo <code>CurrentVoters</code> de cada nodo lista únicamente su propio id (<code>kaf01</code> → id 1, <code>kaf02</code> → id 2, <code>kaf03</code> → id 3), nunca a los otros 2 — cada nodo opera como un mini-clúster aislado de 1 solo miembro.	obligatoria #1 en §5.
WARN	Sin unidad <code>systemd</code> con nombre 'kafka'	El script busca unidades <code>systemd</code> llamadas literalmente 'kafka' y no las encuentra en ninguno de los 3 nodos; el contenedor se administra directamente vía Podman sin una unidad <code>systemd</code> visible que lo supervise.	OBLIGATORIO — confirmar el mecanismo real de arranque persistente (unidad <code>systemd</code> equivalente o gestión Podman con reinicio automático) antes de producción.
WARN	Política de reinicio del contenedor: 'no'	En los 3 nodos, <code>podman inspect</code> reporta política de reinicio <code>no</code> . Ante un crash del proceso Kafka o un reinicio del host, el contenedor no se recupera solo.	OBLIGATORIO — ver acción obligatoria #2 en §5 (configurar <code>always</code> o gestión <code>systemd</code> equivalente).
WARN	<code>node-exporter</code> no detectado (puerto 9100)	El agente de métricas de nodo no responde en :9100 en ninguno de los 3 nodos.	Ver nota aclaratoria a continuación — no bloqueante, mismo tratamiento que en MinIO y Flink.
INFO	JDK 21 funcional, no es la versión homologada (25)	Los 3 nodos corren OpenJDK 21.0.8 dentro del contenedor. El script lo marca PASS por ser compatible con Kafka 4.x (mínimo Java 17), pero no coincide con la versión homologada del proyecto (Java 25).	RECOMENDADO : planificar la actualización a JDK 25 en una ventana de mantenimiento, sin bloquear el uso actual.
INFO	Defecto de script — línea suelta "error" en salida de <code>pbigd-kaf02</code> (sección 8.2)	Solo en el log de <code>pbigd-kaf02</code> , entre la línea <code>[INFO] JVM en contenedor</code> Kafka: ... y el <code>[PASS]</code> siguiente, aparece una línea suelta <code>error</code> sin prefijo de nivel. El valor real de la JVM sí se capturó correctamente (el PASS subsiguiente lo usa), por lo que no afecta el resultado del check — es ruido residual del fallback <code> echo "error" en la línea 561 de validate_kafka_principal.sh.</code>	OPCIONAL : revisar en el script por qué el fallback de captura de <code>java -version</code> imprime ruido adicional al stream de salida sin afectar el resultado; no es un hallazgo de infraestructura.

Nota aclaratoria — contradicción entre alcanzabilidad TCP y registro en el quórum de brokers

La sección 6.3 del propio script concluye `[PASS] Nodos del clúster Principal: 3/3 – completo,`

verificando únicamente que el puerto TCP 9092 responde en los 3 nodos. En el mismo run, las secciones 5.3 y 6.2

reportan `[FAIL] Brokers activos insuficientes: 1` y `[FAIL] Quórum de brokers: 1/3`. Esto no se

resuelve descartando ninguno de los dos resultados: ambos son correctos y describen capas distintas. El puerto 9092

está efectivamente arriba y aceptando conexiones TCP en los 3 nodos (cada broker individual funciona), pero la

alcanzabilidad de un puerto no implica pertenencia al mismo clúster lógico. La causa confirmada (ver

diagnostico_quorum_incompleto.md) es que cada nodo se formateó con un `cluster.id` distinto

(ppGWqJhuQNSaezGXt4ECAg en kaf01, VP6nnxobRa-pAdN7NtBFbQ en kaf02, 7HmRjdAiQrSj8G_3zYdgfw en

kaf03) — Kafka nunca permite que nodos con `cluster.id` distinto se unan al mismo quórum, aunque estén en red

y respondiendo individualmente. El hallazgo de quórum permanece como FAIL real en la matriz; no se trata de un

falso positivo.

Nota aclaratoria — node-exporter (puerto 9100)

Durante la ejecución del script se generó una advertencia indicando que node-exporter no fue detectado en el puerto

9100. Esta advertencia queda desestimada y reclasificada como informativa, en línea con lo confirmado en las

validaciones de MinIO y Flink del mismo proyecto: la instalación no contempla node-exporter como proceso

independiente. Grafana Alloy — activo y confirmado en los 3 nodos — incluye el componente

`prometheus.exporter.unix`, que replica de forma nativa la funcionalidad completa de node-exporter. No existe

brecha de monitoreo de infraestructura.

4. ANÁLISIS TÉCNICO DESTACADO

4.1 Infraestructura base — homogénea entre los 3 nodos

Los 3 brokers comparten el mismo stack de software (Rocky Linux 10.2, Podman 5.8.2, systemd activo, NTP sincronizado), consistente con el patrón observado en MinIO y Flink del mismo proyecto. La sincronización NTP es especialmente relevante aquí: el consenso KRaft usa Raft, sensible a desfases de reloj, y los 3 nodos la confirman correcta — descarta esa hipótesis como causa de los fallos de quórum.

4.2 Persistencia y almacenamiento

`/data/kafka` existe en los 3 nodos sobre filesystem `xfs` (no overlay), montado en una partición `/data` independiente del disco raíz — el diseño de persistencia está correctamente implementado y protege contra la pérdida de datos ante recreación del contenedor.

4.3 Red y conectividad

La capa de red está completamente sana: los 3 nodos se alcanzan entre sí por ping y por TCP en los puertos 9092 (broker) y 9093 (controller). Esto confirma que ningún firewall ni regla de red está bloqueando el tráfico entre nodos — el problema de quórum no es de conectividad, es de identidad de clúster (ver §4.4).

4.4 Quórum KRaft y funcionalidad — causa raíz del NO APTO

El hallazgo central de este informe es que los 3 nodos operan como 3 clústeres KRaft aislados de 1 miembro cada uno, en lugar de un único clúster de 3. La evidencia es consistente en las 3 fuentes: (a) cada nodo reporta solo 1 broker

registrado, (b) cada nodo tiene un `ClusterId` distinto en la salida de `kafka-metadata-quorum.sh`, y (c) `CurrentVoters` en cada nodo lista únicamente su propio id. La causa es que cada nodo se formateó de forma independiente con `kafka-storage.sh format`, generando su propio UUID de clúster en vez de compartir uno solo — `node.id` y `controller.quorum.voters` están correctamente configurados en los 3 (se descartó esa hipótesis), pero eso no importa si el `cluster.id` no coincide. Esto impide la función core del diseño: un clúster de 3 nodos con RF=3 para tolerar la caída de un broker sin pérdida de disponibilidad.

4.5 Ciclo de vida del contenedor

El contenedor Kafka está corriendo y saludable en los 3 nodos (17-26 horas de uptime), pero sin una unidad `systemd` que lo supervise y con política de reinicio no. Esto significa que, incluso una vez resuelto el problema de quórum, un crash del proceso o un reinicio del host dejaría el clúster caído hasta una intervención manual — un riesgo operativo independiente del hallazgo de quórum que debe cerrarse antes de producción.

5. ACCIONES REQUERIDAS ANTES DE PASO A PRODUCCIÓN

OBLIGATORIA #1 — Reformatear el storage KRaft de los 3 nodos con un único cluster.id compartido

1. Generar un solo UUID de clúster (una sola vez, desde cualquier nodo): `kafka-storage.sh random-uuid`
2. Confirmar que no hay datos reales en los topics actuales (el clúster nunca llegó a operar con RF=3).
3. Detener los 3 contenedores Kafka (`kafka-broker-01`, `-02`, `-03`).
4. Limpiar el contenido de `/data/kafka` en los 3 nodos.
5. Reformatear los 3 nodos con el mismo UUID: `kafka-storage.sh format -t <UUID_COMPARTIDO> -c /opt/kafka/config/server.properties`
6. Reiniciar los 3 contenedores.
7. Re-ejecutar `validate_kafka_principal.sh` en los 3 nodos: `CurrentVoters` debe mostrar los 3 ids, "Brokers detectados" debe ser 3, y la creación del topic de prueba con RF=3 debe pasar.

OBLIGATORIA #2 — Configurar arranque persistente y auto-recuperación del contenedor

1. Confirmar o crear la unidad `systemd` que debe supervisar cada contenedor Kafka (o adoptar `podman generate systemd` sobre los contenedores actuales).
2. Configurar política de reinicio `always` a nivel de Podman o `Restart=always` a nivel de la unidad `systemd`, en los 3 nodos.
3. Confirmar que la unidad queda `enabled` para sobrevivir un reinicio de host.

RECOMENDADA — Alinear JDK a la versión homologada

Actualizar de OpenJDK 21.0.8 a Java 25 (versión homologada del proyecto) en los 3 nodos, en una ventana de mantenimiento — Kafka 4.x es funcional hoy con JDK 21, no es bloqueante.

OPCIONAL — Ajustes menores

- Revisar el fallback de captura de `java -version` en la línea 561 de `validate_kafka_principal.sh` para eliminar el ruido "error" residual observado en `pbigd-kaf02`.

6. CONCLUSIÓN

El Clúster Kafka Principal de la plataforma Big Data de la Cooperativa Jardín Azuayo completó la validación de

implementación con resultado **NO APTO / RECHAZADO**. La infraestructura base — sistema operativo, Podman, persistencia en disco, red y observabilidad — está correctamente desplegada y es homogénea en los 3 nodos, y el veredicto negativo declarado por el propio script de validación se confirma con evidencia sólida, no con un falso positivo.

El hallazgo bloqueante es que los 3 nodos nunca se unieron al mismo clúster KRaft: cada uno se formateó con un `cluster.id` propio, por lo que operan como 3 clústeres aislados de 1 miembro cada uno. Esto impide la función core del diseño — un clúster de 3 brokers con factor de replicación 3 — y se confirma con el rechazo consistente de Kafka a crear topics con RF=3 en los 3 nodos. A esto se suma la ausencia de una política de reinicio y de supervisión systemd, que deja al clúster sin auto-recuperación ante una caída futura.

El clúster no puede considerarse apto para producción hasta reformatear el storage KRaft de los 3 nodos con un cluster.id compartido y confirmar, mediante re-validación, un quórum de 3/3 brokers con creación exitosa de un topic RF=3; adicionalmente debe configurarse arranque persistente y política de reinicio always antes de operar el clúster con datos reales.

Elaborado por	Revisado por
---------------	--------------

Bluepoint AI — Consultoría Big Data
Proyecto Big Data — Cooperativa Jardín Azuayo

Equipo de Infraestructura
Cooperativa Jardín Azuayo

Fecha: 09 de julio de 2026

Fecha: _____